

Flujos conversacionales e integración

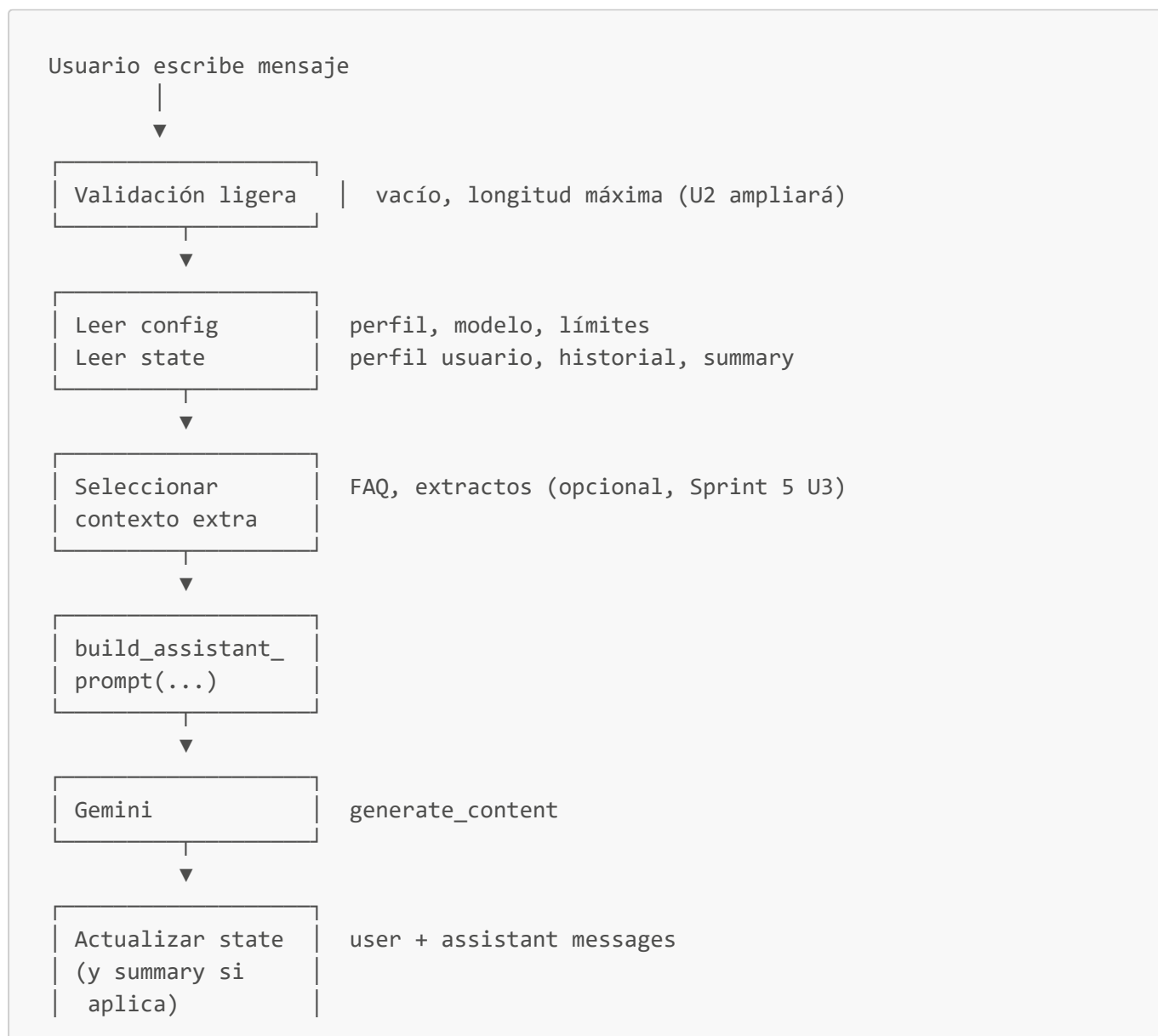
Cada turno es el mismo pipeline: validar → ensamblar contexto → construir prompt → llamar → responder → actualizar estado.

Este documento cierra la unidad: flujo conversacional, integración de prompts y contexto, y el proyecto **Smart Study Assistant** como asistente completo.

Objetivos

- Describir el flujo completo de un turno conversacional.
- Integrar `assistant_config`, `user_state` y contexto seleccionado en un solo prompt.
- Orquestrar desde `logic.py` sin inflar `main.py`.
- Preparar el terreno para Unidad 02 (robustez): ver qué superficies ataca el usuario.

1) Flujo de un turno





Respuesta al usuario

Pseudocódigo en `logic.py`

```
def procesar_turno(
    state: dict,
    config: dict,
    user_message: str,
    extra_context: list | None = None,
) -> dict:
    if not user_message.strip():
        return respuesta_error("Mensaje vacío", ["El mensaje no puede estar vacío."])

    prompt = build_assistant_prompt(
        assistant_config=config,
        user_state=state,
        user_message=user_message,
        extra_context=extra_context or [],
        recent_messages=ultimos_n(state, config["max_turnos_historial"]),
    )

    texto = llamar_gemini(prompt, temperature=config["temperature"])

    append_user(state, user_message)
    append_assistant(state, texto)

    return respuesta_ok("Turno completado", {"respuesta": texto})
```

Orden importante: en el prompt del turno actual, el historial reciente suele ir **sin** duplicar el mensaje que aún no has guardado — el patrón exacto se muestra en el proyecto demo.

2) Integración de prompts y contexto

Sprint 5 te dejó dos piezas:

- **Prompt Engineering:** rol, tarea, plantillas (`build_prompt`).
- **Context Engineering:** qué datos incluir, ventana, resumen, tokens.

En un asistente, la función unificada puede verse así:

```
def build_assistant_prompt(
    *,
    assistant_config: dict,
    user_state: dict,
    user_message: str,
```

```

        extra_context: list,
        recent_messages: list,
    ) -> str:
        perfil = PERFILES[assistant_config["perfil_activo"]]
        profile = user_state.get("user_profile", {})
        summary = user_state.get("summary", "").strip()

        historial_txt = "\n".join(
            f"{m['role']}: {m['content']}" for m in recent_messages
        ) or "(sin turnos previos en la ventana)"

        contexto_txt = "\n\n".join(extra_context) if extra_context else "(sin contexto adicional)"

        return f"""
{perfil["rol"]}

Instrucciones del asistente de estudio:
- Responde en {assistant_config["idioma_respuesta"]}.
- Máximo aproximado: {assistant_config["max_palabras"]} palabras.
- Si no sabes algo, dilo con honestidad.

Perfil del usuario:
- Nombre: {profile.get("nombre") or "(desconocido)}"
- Nivel: {profile.get("nivel", "junior")}
- Tema: {profile.get("tema_actual") or "(sin tema fijado)"}

Resumen de sesión (si existe):
{summary or "(sin resumen aún)"}

Contexto de referencia seleccionado:
{contexto_txt}

Historial reciente:
{historial_txt}

Mensaje actual del usuario:
{user_message.strip()}
""".strip()

```

No necesitas enviar **todo** el FAQ ni **todo** el historial.

3) Flujos conversacionales típicos

A) Onboarding

1. Usuario: "Me llamo Ana y estudio Context Engineering."
2. El asistente responde y Python actualiza `user_profile`.
3. Turnos siguientes usan ese dato sin volver a preguntar.

B) Pregunta con contexto de referencia

1. Usuario pregunta sobre “embeddings”.
2. **logic** selecciona 1 entrada de FAQ (keywords, sin embeddings vectoriales).
3. El prompt incluye solo esa entrada, no el JSON entero.

C) Conversación larga

1. Tras N turnos, generas **summary**.
2. Sigüientes prompts usan **summary + ventana** en lugar del historial completo.

Estos tres escenarios son las **demos del proyecto** [03_proyecto_asistente_estudio_ejemplo/](#).

4) Proyecto integrado: Smart Study Assistant

Estructura del proyecto:

```
03_proyecto_asistente_estudio_ejemplo/  
├── README.md  
├── requirements.txt  
├── .gitignore  
├── .env.example  
├── config.py  
├── state.py  
├── prompts.py  
├── gemini_auth.py  
├── gemini_client.py  
├── logic.py  
├── context.py          # selección FAQ (opcional, de U3)  
├── data/  
│   └── faq.json  
└── main.py
```

Escenas de **main.py**

1. **Comparar perfiles** — misma pregunta, distinto **perfil_activo**.
2. **Sesión con memoria** — presentación + “¿cómo me llamo?”.
3. **Turno con FAQ** — consulta acotada con contexto seleccionado.

Ejecución:

```
python -m venv .venv  
source .venv/bin/activate    # Windows: .venv\Scripts\Activate.ps1  
pip install -r requirements.txt  
cp .env.example .env      # o getpass al ejecutar  
python main.py
```

5) Respuestas estructuradas de la app:

```
def respuesta_ok(mensaje: str, data: dict | None = None) -> dict:
    return {"status": "ok", "mensaje": mensaje, "data": data or {}}

def respuesta_error(mensaje: str, errores: list[str]) -> dict:
    return {"status": "error", "mensaje": mensaje, "data": {"errores": errores}}
```

`main.py` imprime; `logic.py` devuelve dicts. Facilita tests y la Unidad 02 (rechazar turnos inválidos antes del modelo).

6) Puente hacia Robustez y seguridad (Unidad 02)

En esta unidad el usuario escribe **en el mismo canal** que las instrucciones del asistente. Eso es conveniente para aprender, pero **riesgoso** en producción.

Preguntas que deberías hacerte al cerrar la unidad:

- ¿Qué pasa si el usuario escribe “ignora instrucciones anteriores”?
- ¿Dónde viven las reglas inmutables del sistema?
- ¿Validas longitud y contenido sospechoso **antes** de llamar a Gemini?

La Unidad 02 añade capas sin reescribir la arquitectura que has construido aquí.

7) Errores frecuentes

1. **Todo en `main.py`** — demos que no escalan.
 2. **Olvidar actualizar `user_profile`** cuando el usuario se presenta en lenguaje natural (puedes extraer en el turno o pedir formato explícito en MVP).
 3. **No acotar tokens** — historial completo siempre (regresión respecto a Sprint 5, U3).
 4. **Llamar a Gemini antes de validar** — gasto y superficie de ataque innecesarios.
 5. **Copiar el proyecto de Context Engineering sin capa `assistant_config`** — pierdes la distinción producto vs sesión.
-

Resumen

- Un turno = pipeline fijo en `logic.py`.
- `build_assistant_prompt` integra config, estado, contexto seleccionado e historial.
- Hemos visto un asistente de estudio personalizado de turno completo.
- En la siguiente unidad veremos como hacer que el asistente sea más robusto y seguro.